



IIC2343 - Arquitectura de Computadores (I/2026)

Actividad de programación

Sección 1 - Pauta de evaluación

Pregunta 1: Explique el código (3 ptos.)

En el siguiente fragmento de código se realiza el llamado de una subrutina `func_n`:

```
.data
n:      .word 5
res:    .word 0
.text
main:
    la t0, n
    la t1, res
    lw a0, 0(t0)
    addi sp, sp, -4
    sw ra, 0(sp)
    jal ra, func_n
    lw ra, 0(sp)
    addi sp, sp, 4
    sw a0, 0(t1)
    addi a7, zero, 10
    ecall
func_n:
    addi sp, sp, -8
    sw ra, 0(sp)
    sw t2, 4(sp)
    add t2, a0, zero
    beq a0, zero, end
    addi a0, a0, -1
    jal ra, func_n
    add a0, t2, a0
end:
    lw ra, 0(sp)
    lw t2, 4(sp)
    addi sp, sp, 8
    jalr zero, 0(ra)
```

Este fragmento representa el cómputo de una función $f(n)$. A partir de este:

1. (1.5 ptos.) Indique, con argumentos y en términos de n , lo que retorna la función $f(n)$. Por ejemplo, $f(n) = n^2$. Se otorgan **0.75 ptos.** por la correctitud de la descripción del retorno y **0.75 ptos.** por justificación.

Solución: La función computa recursivamente la suma triangular de n , *i.e.* $f(n) = n + n - 1 + \dots + 1 = \frac{n \times (n+1)}{2}$. Esto se evidencia en el hecho de que el valor de retorno, almacenado en el registro `a0`, es igual a $n + f(n - 1)$, con caso base $f(0) = 0$. De esta forma, se tiene que:

$$f(n) = n + f(n - 1) = n + n - 1 + f(n - 2) = \dots = n + n - 1 + \dots + 1 = \frac{n \times (n + 1)}{2}$$

No es necesario que se haga un detalle completo del código, pero sí que se demuestre que se entiende el cómputo recursivo de la suma triangular de n .

2. (1.5 ptos.) Indique, con argumentos, si el fragmento anterior respeta o no la convención de llamadas de RISC-V. Se otorgan **0.75 ptos.** si indica de forma correcta si se respeta o no la convención y **0 ptos.** si su respuesta es incorrecta. Por otra parte, se otorgan **0.75 ptos.** por entregar una justificación válida respecto a su respuesta.

Solución: El fragmento anterior **sí respeta** la convención de llamadas de RISC-V. Los motivos son los siguientes:

- Se respalda `ra` antes de cada llamado.
- Se usa el registro `a0` para el argumento de `func_n`, y `a0` para su retorno.
- Se respalda el registro `t2` **antes** del llamado recursivo de la subrutina, considerando su sobreescritura; su uso posterior al llamado; y el hecho de ser *caller-saved*. Aquí es importante notar que esto es correcto porque es *caller-saved* sobre el llamado recursivo, no sobre el inicial (donde correctamente no se respalda `t2` al no utilizarse fuera del *scope* de la subrutina).

Pregunta 2: Elabore el código: Gemelos lejanos (3 ptos.)

Elabore, utilizando el Assembly RISC-V, un programa que a partir de un arreglo `arr` de largo `len`, determine la **máxima distancia** (*i.e.* la diferencia de índices) entre dos elementos que posean **exactamente** el mismo valor. El valor de esta distancia máxima debe ser almacenado en la variable `res`; en el caso de que no existan números repetidos, el valor de la variable debe ser igual a 0. Puede asumir que $\text{len} \geq 1$.

A continuación, tres ejemplos:

```
arr = [3, 2, 1, 2, 4, 3]  len = 6 → res = 5
arr = [1, 1, 1]          len = 3 → res = 2
arr = [1, 2, 3, 4]       len = 4 → res = 0
```

Puede utilizar el siguiente fragmento de código como base:

```
.data
arr: .word 3, 2, 1, 2, 4, 3
len: .word 6
res: .word -1
.text
# Su código aquí.
# Puede utilizar este label para terminar el programa
end_program:
    addi a7, zero, 10
    ecall
```

La asignación de puntaje se distribuirá de la siguiente forma:

- **1 pto.** por resolver correctamente el caso donde no existen elementos repetidos en el arreglo (retornando 0). Se descuentan **0.5 ptos.** si existe como máximo un error de implementación y no se asigna puntaje si existe más de uno.
- **2 ptos.** por resolver correctamente el caso donde sí existen valores repetidos, recorriendo el arreglo de forma anidada o con múltiples punteros para calcular y actualizar la mayor distancia posible. Se descuenta **1 pto.** si existe como máximo un error de implementación y no se asigna puntaje si existe más de uno.

IMPORTANTE: No es necesario que respete la convención en este ejercicio.

Tips

- No es necesario que haga el cómputo a través de subrutinas. Puede hacerlo de manera iterativa con saltos condicionales.
- Para obtener la dirección del i -ésimo elemento de un arreglo en RISC-V, puede usar esta fórmula: $\text{dir}(\text{arr}[i]) = \text{dir}(\text{arr}) + 4 \times i$.

Solución: A continuación, se muestra una solución que realiza el siguiente procedimiento iterativo:

- Inicia la iteración con $i = 0$ y $j = 1$.
- Si $\text{arr}[i] == \text{arr}[j]$, se revisa si la distancia actual es mayor a la última calculada.
- Solo si es mayor, se actualiza la máxima distancia encontrada. Luego, continua el ciclo con $j++$.
- Si $j \geq \text{len}$, entonces sigue la iteración con $i++$ y $j = i + 1$, es decir, revisamos siempre desde i hacia adelante.
- Si $i \geq \text{len} - 1$, ya se revisaron todas las combinaciones y, por ende, termina el programa actualizando la variable `res` con el máximo encontrado, almacenado en `t2`.

```
.data
arr: .word 3, 2, 1, 2, 4, 3
len: .word 6
res: .word -1
.text
init:
    lw s0, len                # s0 = Largo arr
    addi s1, s0, -1           # s1 = Largo arr - 1
    la s2, res                 # s2 = Dir. memoria res
    la s3, arr                 # s3 = Dir. memoria arr[i]
    addi t0, zero, 0           # t0 = i = 0
    addi t1, zero, 1           # t1 = j = 1
    beq s1, zero, end_program  # if s1 == 0 -> len = 1, termina programa con máxima dist. = 0

loop:
    add t3, zero, t0           # t3 = i
    slli t3, t3, 2              # t3 = 4*i (2 shift lefts)
    add t3, t3, s3              # t3 = dir(arr) + 4*i = dir(arr[i])
    lw t3, 0(t3)               # t3 = arr[i]

    continue_loop:
        add t4, zero, t1        # t4 = j
        slli t4, t4, 2          # t4 = 4*j (2 shift lefts)
        add t4, t4, s3          # t4 = dir(arr) + 4*j = dir(arr[j])
        lw t4, 0(t4)           # t4 = arr[j]
        bne t3, t4, next_j      # if t3 != t4 -> no gemelos, avanza j
        sub t5, t1, t0          # t5 = j - i -> distancia entre gemelos
        ble t5, t2, next_j      # if t5 <= t2 -> distancia <= a la máxima actual, avanza j
        add t2, zero, t5        # t2 = t5 -> nueva distancia máxima

    next_j:
        addi t1, t1, 1          # j += 1
        bge t1, s0, next_i      # if t1 >= s0 -> j >= len, avanza i y restituye j
        jal zero, continue_loop # Reinicia loop manteniendo arr[i], ya que no cambia

    next_i:
        addi t0, t0, 1          # i += 1
        bge t0, s1, end_program # if t0 >= s1 -> i >= len - 1, termina programa (último elemento)
        addi t1, t0, 1          # j = i + 1, solo vemos las combinaciones hacia adelante
        jal zero, loop          # Reinicia loop desde el comienzo

end_program:
    sw t2, 0(s2)                # res = distancia máxima encontrada, o cero
    addi a7, zero, 10
    ecall
```

Aparte de la distribución de puntaje antes señalada, se realizan las siguientes consideraciones:

- Si no se llega a la solución correcta en ningún caso, o bien el programa se cae, se otorgarán **0.75 ptos.** si se evidencia a nivel de código un intento de algoritmo cercano a lo esperado.
- Si el programa falla **solo** en el caso donde `len == 1`, se descuentan solo **0.5 ptos.** del total.